Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Уральский федеральный университет
имени первого Президента России Б.Н. Ельцина»
Институт новых материалов и технологий
Кафедра «Теплофизика и информатика в металлургии»

Управление контейнерами с помощью Docker Compose и использование готовых образов

ОТЧЕТ
по практической работе № 2
по дисциплине «Основы методологии Development Operation»

Направление 09.03.02 «Информационные системы и технологии» (уровень бакалавриата)
Образовательная программа
09.03.02/33.02 «Информационные системы и технологии» (СУОС)

Студент
группы НМТ-433901 Малышева О.М.

Преподаватель:
профессор, д.т.н. В.В. Лавров

Екатеринбург
2026

СОДЕРЖАНИЕ

1.1 Цель работы 3
1.2 Ход проведения работы 3
1.2.1 Разработка тестового приложения Visual Studio .NET Core 3
1.2.2 Создание конфигурационного файла docker-compose.yml 3
1.2.3 Размещение приложения в системе контроля версий GitHub 5
1.2.4 Сборка образов с использованием команды docker compose 5
1.2.5 Установка программы HeidiSQL для сопровождения базы данных в СУБД MariaDB 7
1.2.6 Настройка миграции базы данных в MariaDB 8
1.2.7 Демонстрация работы приложения с базой данных в СУБД MariaDB 8
1.3 Выводы 9
Приложение А. Листинг программного кода приложения «Калькулятор» (контроллер) 10
Приложение Б. Листинг программного кода приложения «Калькулятор» (appsettings.json) 11
Приложение В. Листинг программного кода приложения «Калькулятор» (appsettings.Development.json) 12
Приложение Г. Листинг программного кода приложения «Калькулятор» (Program.cs) 13
Приложение Д. Фрагмент конфигурационного файла docker-compose.yml 14

1.1 Цель работы

Цель практической работы — настроить совместный запуск веб-приложения «Калькулятор» и системы управления базами данных MariaDB с использованием Docker Compose, проверить взаимодействие приложения с базой данных и разместить исходный код проекта в GitHub.

В ходе работы необходимо подготовить конфигурационный файл docker-compose.yml, собрать и запустить контейнеры, выполнить настройку подключения к MariaDB, проверить сохранение истории вычислений и зафиксировать результаты с помощью скриншотов.

1.2 Ход проведения работы

Работа выполнялась поэтапно: сначала проверена структура проекта и интерфейс калькулятора, затем подготовлена конфигурация Docker Compose для веб-приложения и базы данных. После этого проверены контейнеры, подключение к MariaDB и сохранение вычислений.

1.2.1 Разработка тестового приложения Visual Studio .NET Core

В Visual Studio используется веб-приложение «Calculator», реализованное на ASP.NET Core. Пользователь вводит два числа, выбирает арифметическую операцию и нажимает кнопку «Рассчитать». После выполнения операции на странице отображается результат, а пользователь может открыть историю вычислений.

Для хранения истории используется таблица Calculations, в которой представлены идентификатор записи, первое и второе числа, операция, результат и дата создания. Наличие записей в таблице позволяет проверить, что данные вычислений сохраняются в базе.

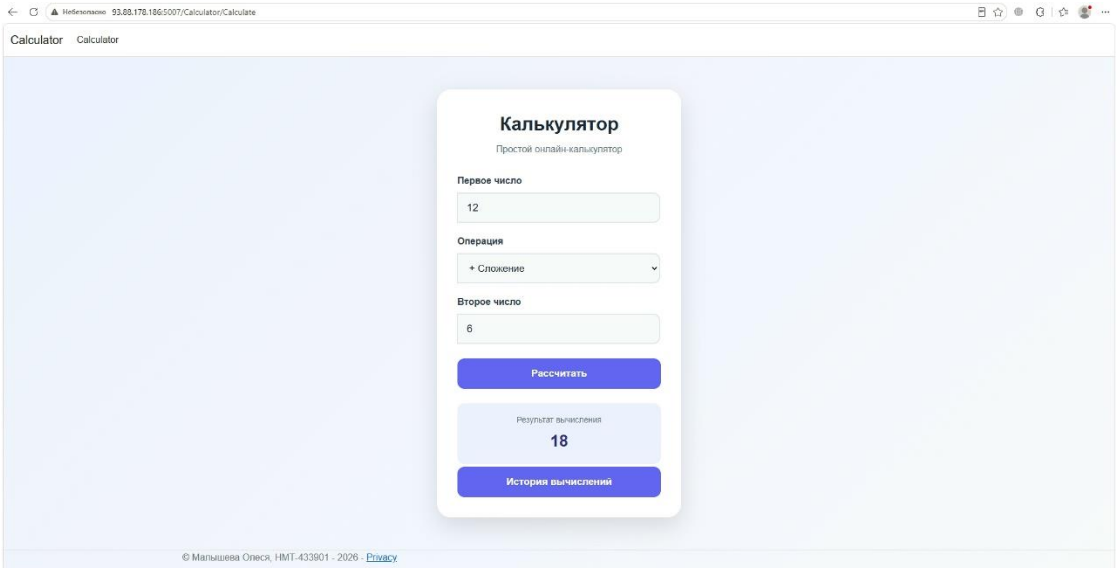


Рисунок 1 — Интерфейс калькулятора с результатом вычисления

1.2.2 Создание конфигурационного файла docker-compose.yml

Для совместного запуска компонентов проекта создан файл docker-compose.yml. В нём определены два сервиса: mariadb и web. Для MariaDB

используется готовый образ mariadb:10.5, задаются имя базы данных, пользователь и пароль, а также публикация порта 5023 на порт 3306 контейнера.

Сервис web собирается из каталога Calculator с использованием расположенного в нём Dockerfile. В конфигурации задано имя контейнера и публикация порта веб-приложения 5015. Такой подход позволяет описать параметры запуска обоих компонентов в одном файле.

docker-compose.yml

C: > Users > Olesia.Malysheva > Documents > GitHub > 433901_devops_Malysheva

```yaml
version: '3.7'

services:
  mariadb:
    container_name: web-mariadb
    hostname: mariadb
    image: mariadb:10.5
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: password
      MARIADB_DATABASE: webdb
      MARIADB_USER: webdb
      MARIADB_PASSWORD: password
    ports:
      - "0.0.0.0:5023:3306"

  web:
    container_name: web-app-calculator
    hostname: app
    build: Calculator/
    ports:
      - "0.0.0.0:5015:5015"
```

Рисунок 2 — Файл docker-compose.yml с сервисами MariaDB и web

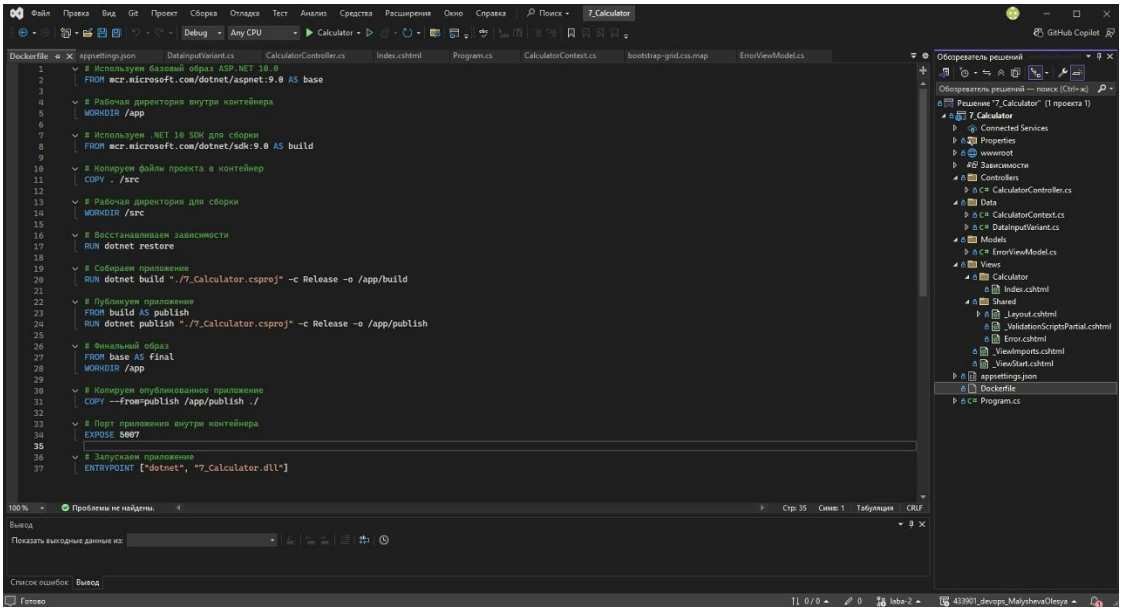


Рисунок 3 — Dockerfile веб-приложения в Visual Studio

1.2.3 Размещение приложения в системе контроля версий GitHub

Исходный код проекта размещён в удалённом репозитории GitHub. На скриншоте репозитория отображаются ветка master и рабочая ветка laba-2. Использование системы контроля версий позволяет сохранять изменения конфигурации и исходного кода, а также отслеживать состояние проекта.

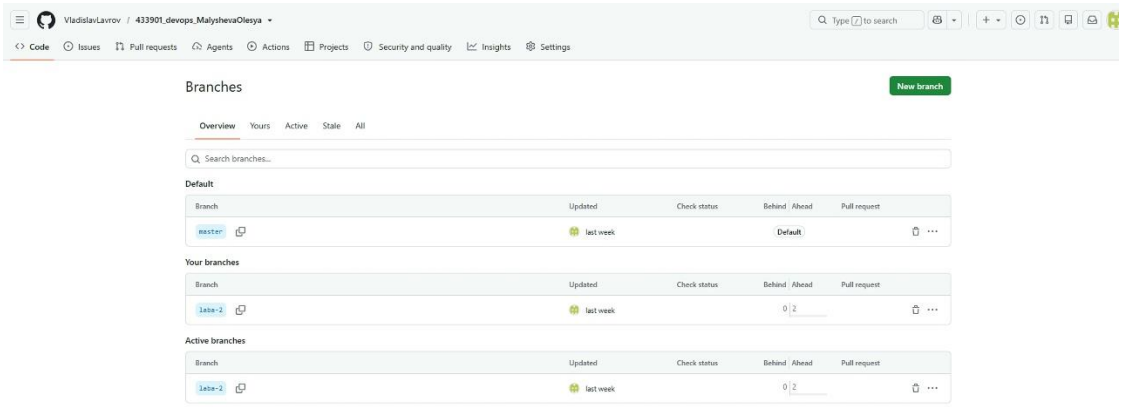


Рисунок 4 — Ветки репозитория GitHub

1.2.4 Сборка образов с использованием команды docker compose

Перед запуском была проверена конфигурация командой docker compose config. Вывод команды показывает итоговые параметры сервисов, включая используемый образ MariaDB, настройки окружения и опубликованные порты.

Затем с помощью Docker Compose были собраны и запущены сервисы проекта. Команда docker ps использовалась для проверки состояния контейнеров и сопоставления опубликованных портов. На скриншоте видны работающие контейнеры веб-приложения и MariaDB.

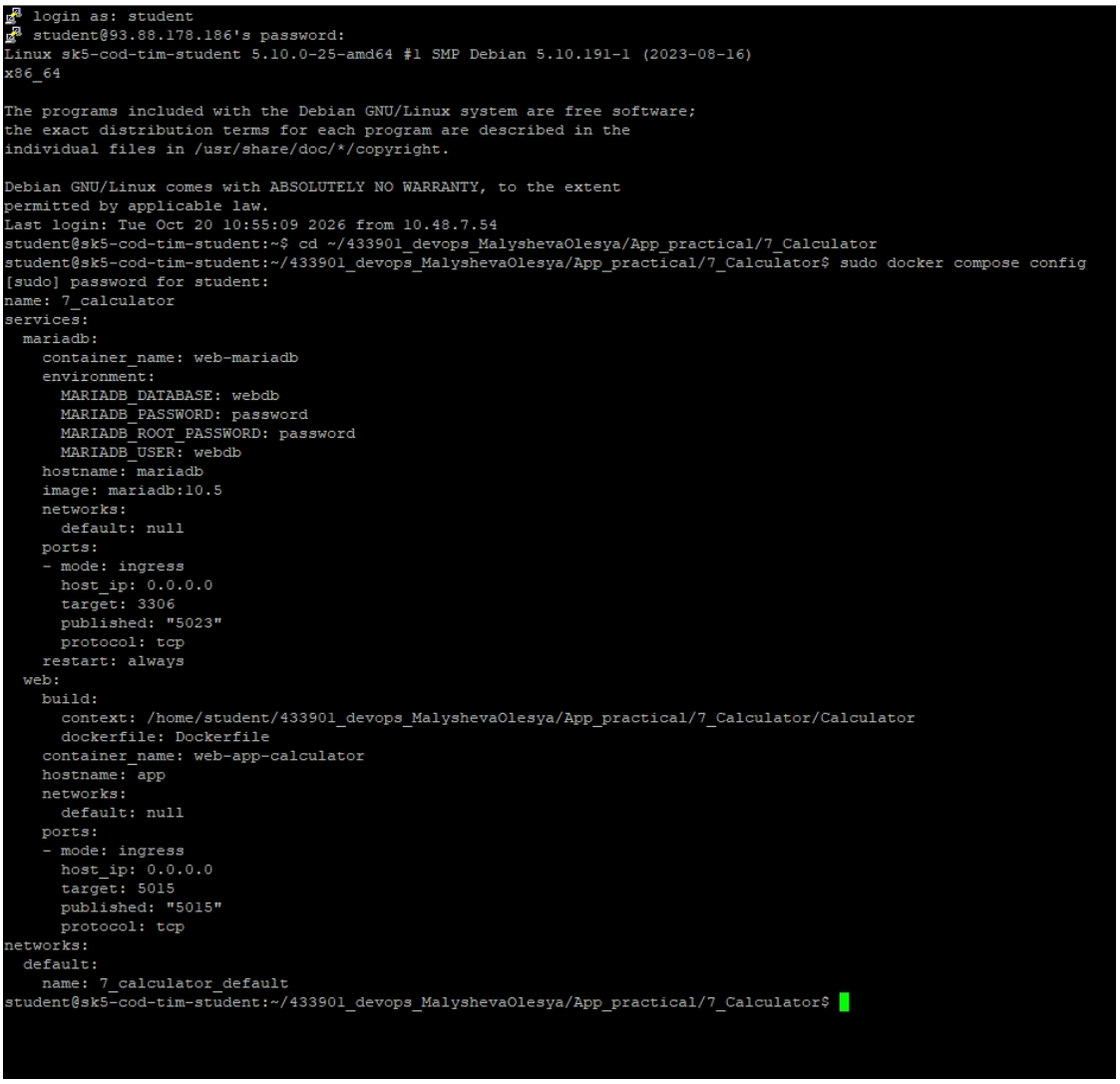


Рисунок 5 — Проверка конфигурации командой docker compose config

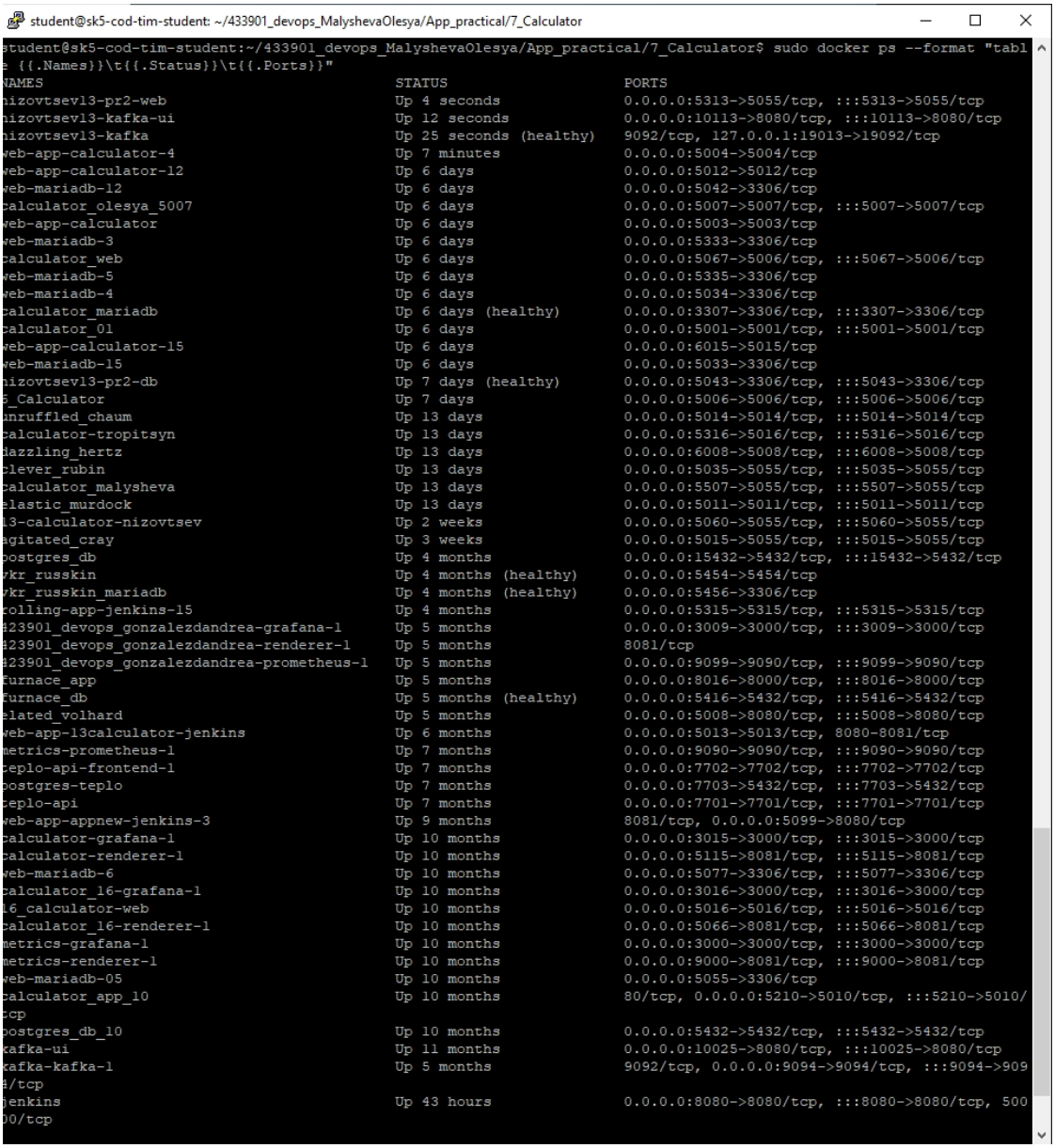


Рисунок 6 — Список контейнеров и опубликованных портов

1.2.5 Установка программы HeidiSQL для сопровождения базы данных в СУБД MariaDB

Для просмотра содержимого базы данных использована программа HeidiSQL. После подключения к серверу MariaDB была открыта база calculator_db. В ней отображаются таблицы __EFMigrationsHistory и Calculations, что подтверждает наличие структуры базы данных и таблицы, предназначенной для хранения истории вычислений.

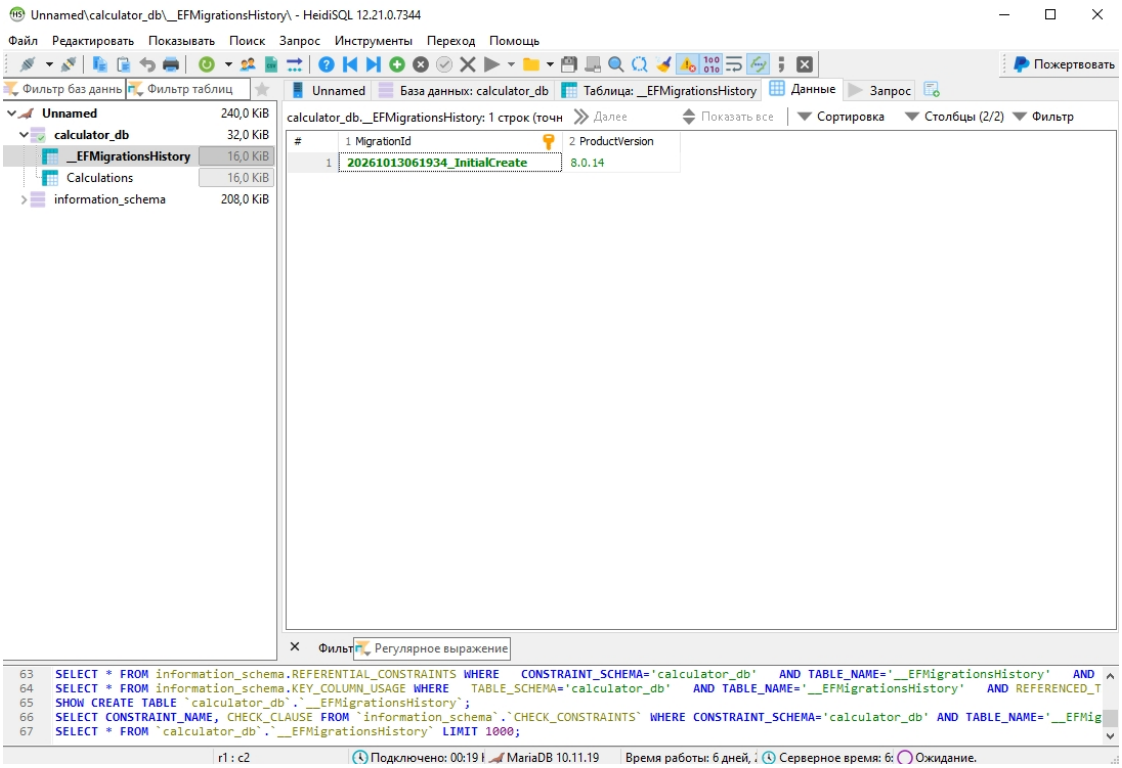


Рисунок 7 — База calculator_db и таблица __EFMigrationsHistory в HeidiSQL

1.2.6 Настройка миграции базы данных в MariaDB

Для создания структуры базы данных используется механизм миграций Entity Framework Core. В таблице __EFMigrationsHistory отображается запись InitialCreate и версия продукта 8.0.14. Это свидетельствует о том, что миграция была зарегистрирована в базе данных.

Проверка таблицы миграций и таблицы Calculations в HeidiSQL позволяет убедиться, что структура базы данных создана и доступна для приложения.

1.2.7 Демонстрация работы приложения с базой данных в СУБД MariaDB

После выполнения вычисления в веб-приложении на странице отображается результат 18 для операции сложения чисел 12 и 6. Ниже доступна история вычислений, в которой отображаются ранее выполненные операции.

В HeidiSQL в таблице Calculations присутствуют соответствующие записи, в том числе операция сложения 12 и 6 с результатом 18. Совпадение данных в интерфейсе приложения и таблице базы данных подтверждает сохранение истории вычислений в MariaDB.

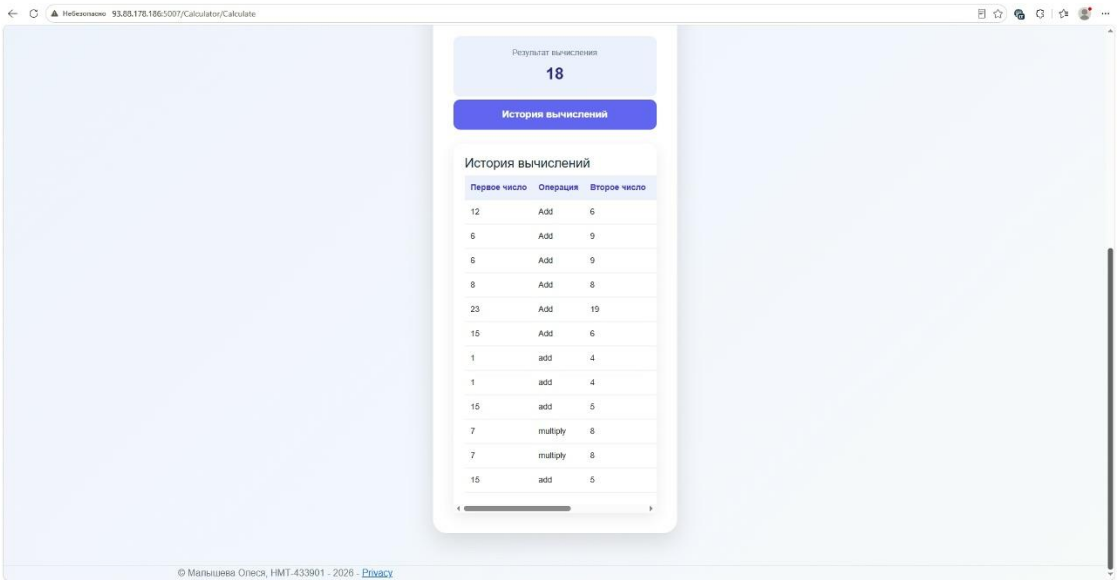


Рисунок 8 — История вычислений в веб-приложении

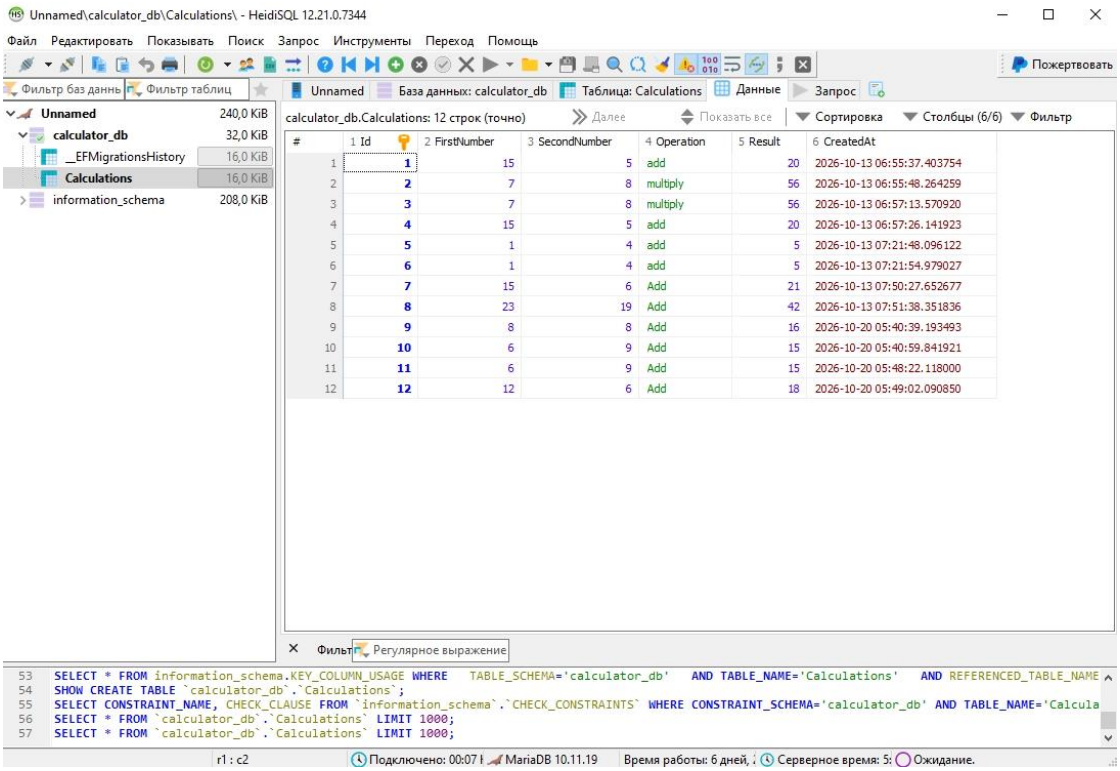


Рисунок 9 — Содержимое таблицы Calculations в HeidiSQL

1.3 Выводы

В ходе практической работы была подготовлена конфигурация Docker Compose для запуска веб-приложения и MariaDB. Проверены параметры сервисов, состояние контейнеров и публикация портов. Исходный код проекта размещён в GitHub.

Работа с HeidiSQL позволила проверить наличие миграции InitialCreate и таблицы Calculations. Выполнение вычисления в веб-приложении и наличие соответствующей записи в базе данных подтверждают взаимодействие приложения с MariaDB. Поставленная цель достигнута.

Приложение А. Листинг программного кода приложения «Калькулятор» (контроллер)

```
using Calculator.Data;
using Calculator.Models;
using Microsoft.AspNetCore.Components.Forms;
using Microsoft.AspNetCore.Mvc;

namespace Calculator.Controllers
{
    public enum Operation { Add, Subtract, Multiply, Divide }

    public class CalculatorController : Controller
    {
        private CalculatorContext _context;

        public CalculatorController(CalculatorContext context)
        {
            _context = context;
        }

        [HttpGet]
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Calculate(double num1, double num2, Operation operation)
        {
            double result = 0;
            switch (operation)
            {
                case Operation.Add:
                    result = num1 + num2;
                    break;
                case Operation.Subtract:
                    result = num1 - num2;
                    break;
                case Operation.Multiply:
                    result = num1 * num2;
                    break;
                case Operation.Divide:
                    result = num1 / num2;
                    break;
            }
            ViewBag.Result = result;

            DataInputVariant dataInputVariant = new DataInputVariant();
            dataInputVariant.Operand_1 = num1.ToString();
            dataInputVariant.Operand_2 = num2.ToString();
            dataInputVariant.Type_operation = operation.ToString();
            dataInputVariant.Result = result.ToString();

            //_context.DataInputVariants.Add(dataInputVariant);
            //_context.SaveChanges();

            return View("Index");
        }
    }
}
```

Приложение Б. Листинг программного кода приложения «Калькулятор» (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=mariadb; Database=myWebCalculatorDb; Uid=root; Pwd=password;
Character Set=utf8; ConvertZeroDatetime=True;"
  },
  "Kestrel": {
    "Endpoints": {
      "Http": {
        "Url": "http://0.0.0.0:5007" // Заменить последние 2 цифры порта на порядковый номер из
студенческого журнала. Например, порт 5012 соответствует номеру 12
      }
    }
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

Приложение В. Листинг программного кода приложения «Калькулятор» (appsettings.Development.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=93.88.178.186; Port=5023; Database=myWebCalculatorDb; Uid=root;
Pwd=password; Character Set=utf8; ConvertZeroDatetime=True;"
  },
  "DetailedErrors": true,
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

Приложение Г. Листинг программного кода приложения «Калькулятор» (Program.cs)

```
using Microsoft.EntityFrameworkCore;
using Calculator.Data;

var builder = WebApplication.CreateBuilder(args);

// Добавление сервисов
builder.Services.AddControllersWithViews();

string mariadbCS = builder.Configuration.GetConnectionString("DefaultConnection");

builder.Services.AddDbContext<CalculatorContext>(options =>
{
    options.UseMySql(
        mariadbCS,
        new MySqlServerVersion(new Version(10, 5, 15))
    );
});

builder.Services.AddRazorPages();

var app = builder.Build();

// Настройка HTTP-конвейера
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Calculator/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Calculator}/{action=Index}/{id?}");

app.Run();
```

Приложение Д. Фрагмент конфигурационного файла docker-compose.yml

```
version: '3.7'

services:
  mariadb:
    container_name: web-mariadb
    hostname: mariadb
    image: mariadb:10.5
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: password
      MARIADB_DATABASE: webdb
      MARIADB_USER: webdb
      MARIADB_PASSWORD: password
    ports:
      - "0.0.0.0:5023:3306"

  web:
    container_name: web-app-calculator
    hostname: app
    build: Calculator/
    ports:
      - "0.0.0.0:5015:5015"
```

Примечание: листинг набран по представленному скриншоту конфигурации. Перед сдачей следует сверить его с актуальным docker-compose.yml из репозитория.